Product Engineer Test

Search & display of a school, a program and its variants

3 hours · Claude API key provided · Edumapper data provided

> **What we're testing.**
> Your ability to cleanly derive a real data model: a program and its variants into a snappy search and display UI. We look at how you handle the data, whether you use the **right data source**, and whether you respect the **Edumapper design tokens.** No documentation to write: we judge what you build and how you did it.

Some context

Edumapper helps high-school and university students choose their path from a catalog of programs. We work in product cells: one product engineer and one expert in their field, owning a topic from idea to launch.

A single program often exists in several **variants**:

- English track,
- Italian track,
- apprenticeship,
- campus,
- Sometimes it could be a "red team" track.

We decided to model them like an e-commerce catalog: a generic product and its variants.

The model: program → variants

The details are authoritative in PRD-015 (attached). The essentials:

- **Generic** = an `edu_program` row without `parent_program_id`.
- **Variant** = a row with `parent_program_id` pointing to its generic.
- The distinction is **derived** from `parent_program_id`, there is no "type" field.
- The `variants` column (JSON) describes what sets the variant apart; `long_title` is deprecated.

> variants examples
> `{ "language": "en", "apprenticeship": true, "school_id": "S11" }`
> `{ "track": "red team" }`

Your task, 3 hours to do it !

From the provided data, build **a full page/app with a search feature** and **a clean way of displaying** a program and its variants. You have a Claude API key, feel free to use it to code, check the data, do whatever you want !

- **Search**: find a program (be smart, pick what you feel is right as criteria).
- **Display:** a component showing the generic and its variants, making the variation axes (language, apprenticeship, track, campus) immediately readable.
- **Respect the model:** and use the provided data, don't invent programs.
- **Care for the code and the front:** a clean, well-structured Nuxt 4 codebase and a pleasant interface.

What we provide

- `edu_program.csv`: the list of programs (generics + variants)
- `edu_school.csv`: the schools / campuses referenced by `school_id`
- `PRD-015-program-variants.md`: the product decision that defines the model
- `NOTE_data_team.md`: a quick note from the data team (read it first)
- A Claude API key, provided separately.

The Edumapper codes and token

We expect your work on UI to be true to how we design:

- Start from the feature and the goal, never from the layout.
- One primary action; establish hierarchy through contrast; let the format speak.
- Start with too much space; prefer shadow and whitespace over hard borders.
- Never convey information by color alone; care for the invisible: empty, loading, error states, keyboard focus.
- Go on our website and figure out our design token

Constraints & deliverable

- **Time: 3 hours.** A tight, clean scope over an over-engineered mess.
- **Imposed stack: Nuxt** (Nuxt 4 / Vue 3). TypeScript.
- **No tests expected.** Invest that time in clean code and a polished front.
- **Deploy the result** somewhere public (Vercel, Netlify, Cloudflare…) and share the URL + the Git repo.
- **Written notes: 5 to 8 lines max**, your though, how it went and what you chose not to do.
- Be inventive, creative and if you think something is missing, implement it !

Remember ! If you succeed, we'll walk through your submission during the interview: how you distinguish generic from variant, what your screen does when facing dirty data, where you used Claude and where you took back control. Be ready to explain your choices.